

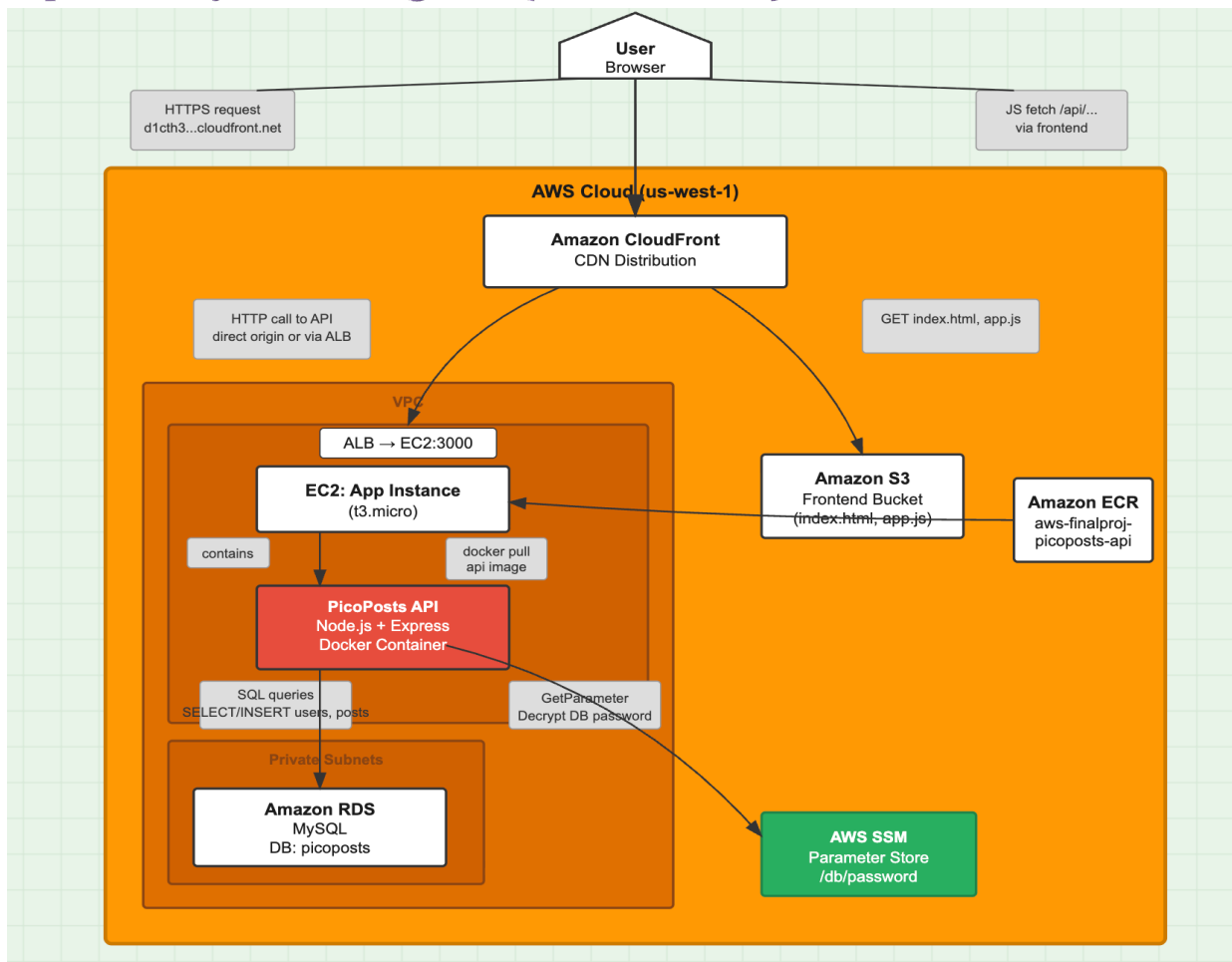
Commit # - [3ebb489](#) · [Git Repo](#)

1. Introduction

The updated design follows a more realistic, secure, production-style architecture. Compared to my original draft, the main improvements are:

- least privilege IAM
- private networking for RDS
- no long-term credentials
- CloudFront routing improvements
- structured IaC governance

2. Updated System Diagram (Placeholder)



- **Front-end Flow**

When the user views the CloudFront URL, static HTML and React are provided from S3. When a user selects "Sign Up" or "Post," CloudFront sends ALB /api/* requests. ALB forwards to port 3000 on EC2.

- **Backend Processing**

When an EC2 container starts, it uses temporary IAM credentials to retrieve the database password from SSM. Within the private subnet, EC2 is connected to RDS.

- **API manages:**

POST to /api/users

POST to /api/posts

GET /api/feed

- **Flow of Security**

EC2 and RDS are never directly communicated with by users. There is never a public IP for RDS. Code or environment variables are never used to hold secrets.

3. Least Privilege Implementation

3.1 IAM Policy Slimming

The IAM role attached to the EC2 instance originally allowed broad access (e.g., s3:*, rds:*, logs:*). This was reduced to only the permissions required for runtime operations:

- - logs:CreateLogGroup
- - logs:CreateLogStream
- - logs:PutLogEvents
- - secretsmanager:GetSecretValue
- - ssmmessages:CreateControlChannel / CreateDataChannel
- - ec2messages:GetMessages / SendReply
- - ssm:UpdateInstanceInformation

Final minimal IAM policy:

```
{
  "Version": "2008-10-17",
  "Id": "PolicyForCloudFrontPrivateContent",
  "Statement": [
    {
      "Sid": "AllowCloudFrontServicePrincipal",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::picoposts-v2-frontend-us-west-1-daf90a/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::240378375517:distribution/E3BS5YP38OWDK6"
        }
      }
    }
  ]
}
```

IAM > Roles > [picoposts-v2-app-ec2-role](#) > Edit policy

- Step 1
- ☒ **Modify permissions in picoposts-v2-app-ec2-ssm-inline**
- Step 2
- ☐ Review and save

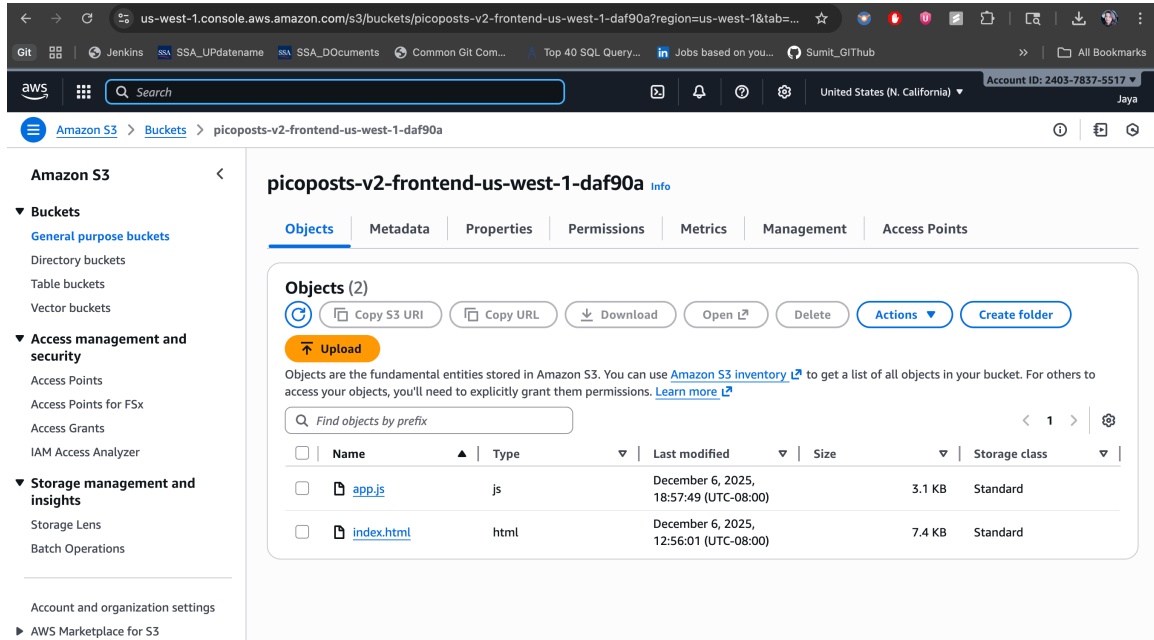
Modify permissions in picoposts-v2-app-ec2-ssm-inline [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Action": [
6         "ssm:GetParameter"
7       ],
8       "Effect": "Allow",
9       "Resource": "arn:aws:ssm:us-west-1:240378375517:parameter/app/picoposts-v2/db/password"
10     }
11   ]
12 }
```

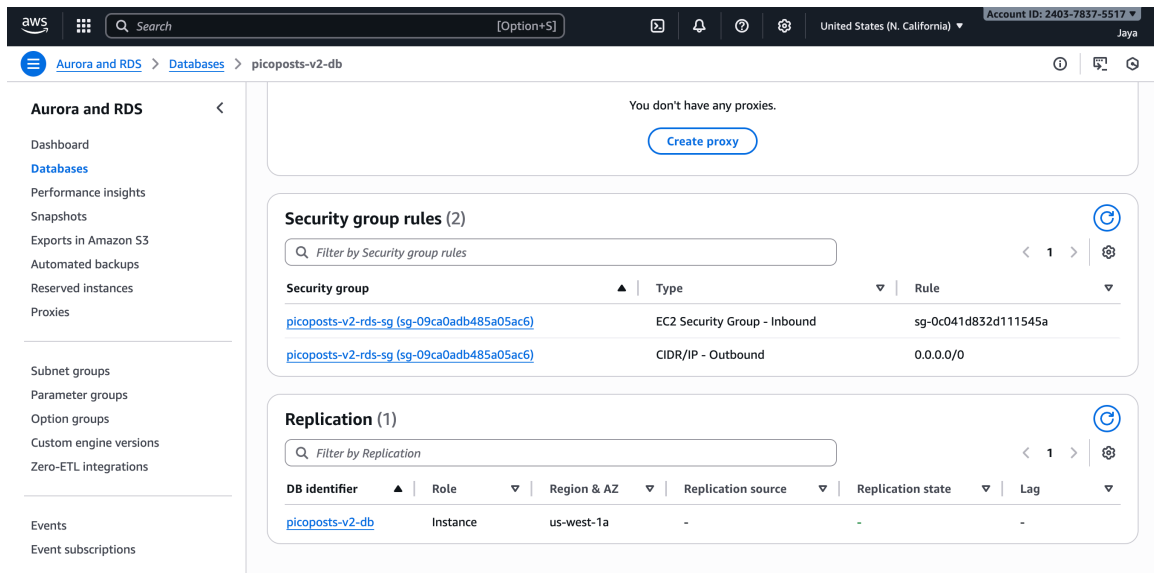
S3 Least-Privilege Controls



The screenshot shows the AWS S3 console interface. The left sidebar contains navigation links for Amazon S3, Buckets, Access management and security, and Storage management and insights. The main content area displays the 'picoposts-v2-frontend-us-west-1-daf90a' bucket. The 'Objects' tab is active, showing a list of two objects: 'app.js' (3.1 KB) and 'index.html' (7.4 KB). The interface includes search bars, filters, and action buttons like 'Upload', 'Download', and 'Delete'.

3.2 Network Segmentation

In order to prevent public access, the RDS instance was set up in private subnets. The database on port 3306 can only be accessed by the security group of the EC2 instance. Internet traffic travels via CloudFront → ALB → EC2 → RDS.



The screenshot shows the AWS Aurora and RDS console interface. The left sidebar contains navigation links for Aurora and RDS, Dashboard, Databases, Performance insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, and Event subscriptions. The main content area displays the 'picoposts-v2-db' database instance. The 'Security group rules' section shows two rules: 'EC2 Security Group - Inbound' and 'CIDR/IP - Outbound'. The 'Replication' section shows one rule for 'picoposts-v2-db'.

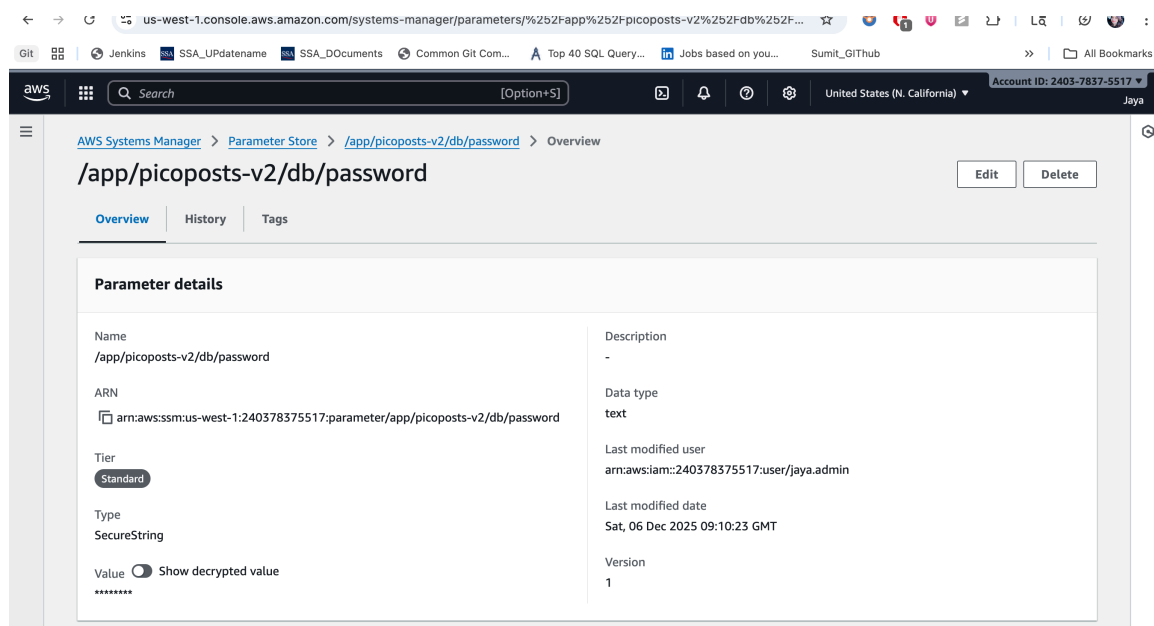
4. Credential Elimination (No Hardcoded Secrets)

Before, my DB password was being passed around manually.

Now:

- Terraform generates the DB password
- Stores in SSM SecureString
- EC2 retrieves it using its IAM role at runtime
- No password appears in Docker, GitHub, Terraform code, or scripts

This was one of the biggest improvements.



5. CloudFront API Routing Fix

Originally, I was calling the backend ALB directly from the browser, which caused CORS issues.

Now CloudFront has:

- Origin 1: S3 (frontend)
- Origin 2: ALB (backend)

Behavior:

- /api/* → ALB
- Everything else → S3

This makes the frontend fully serverless and production-like.

Before/After

Feature	Before	After
IAM Policy	Broad, unnecessary permissions	Minimal: only SSM + CloudWatch
Secrets	Hardcoded DB password	Stored in SSM + IAM temporary access
Networking	RDS could be exposed	RDS fully private, SG-restricted
Frontend Routing	Direct to ALB (CORS issues)	CloudFront routes /api/* securely
Infra Changes	Manual console edits	Terraform-only changes
Tagging	Inconsistent	Enforced through Terraform

5.2 Resource Tagging Policy

Every AWS resource created by Terraform inherits mandatory tags:

Project: PicoPosts
Owner: JayaVyas
Environment: Dev

EC2

Dashboard

AWS Global View

Events

Instances

Instances Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volume

IAM Role

[picoposts-v2-app-ec2-role](#)

IMDSv2

Optional

EC2 recommends setting IMDSv2 to required | [Learn more](#)

Operator

-

Subnet ID

[subnet-0d1e5b4cf2f4076fd \(picoposts-v2-public-us-west-1a\)](#)

Instance ARN

[arn:aws:ec2:us-west-1:240378375517:instance/i-09d25499d4d8bd2ab](#)

Auto Scaling Group name

[picoposts-v2-app-asg](#)

Managed

false

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

Tags

Manage tags

< 1 >

Key	Value
Project	picoposts-v2
Name	picoposts-v2-app-ec2
aws:autoscali...	picoposts-v2-app-asg
aws:ec2launc...	lt-01b3066e7af84494f
aws:ec2launc...	1

6. Application Functionality Screenshots

1. Terraform Apply Output

```
Terminal Local x Local (2) x + v
aws_db_instance.primary: Still creating... [07m10s elapsed]
aws_db_instance.primary: Still creating... [07m20s elapsed]
aws_db_instance.primary: Still creating... [07m30s elapsed]
aws_db_instance.primary: Still creating... [07m40s elapsed]
aws_db_instance.primary: Still creating... [07m50s elapsed]
aws_db_instance.primary: Still creating... [08m00s elapsed]
aws_db_instance.primary: Creation complete after 8m7s [id=db-GL6X72LZQ5NKB5NBNBGVWQ72WY]
aws_launch_template.app: Creating...
aws_launch_template.app: Creation complete after 6s [id=lt-01b3066e7af84494f]
aws_autoscaling_group.app: Creating...
aws_autoscaling_group.app: Still creating... [00m10s elapsed]
aws_autoscaling_group.app: Creation complete after 16s [id=picoposts-v2-app-asg]
aws_autoscaling_attachment.asg_tg: Creating...
data.aws_instances.app: Reading...
data.aws_instances.app: Read complete after 0s [id=us-west-1]
aws_autoscaling_attachment.asg_tg: Creation complete after 0s [id=picoposts-v2-app-asg-20251206091852145700000005]

Apply complete! Resources: 20 added, 0 changed, 1 destroyed.

Outputs:
app_public_ip = "54.241.105.28"
cdn_domain = "d1cth3jm7a29a8.cloudfront.net"
db_primary_endpoint = "picoposts-v2-db.c7uk4uq089w4.us-west-1.rds.amazonaws.com"
db_replica_endpoint = ""
frontend_bucket = "picoposts-v2-frontend-us-west-1-daf90a"
uploads_bucket = "picoposts-v2-uploads-us-west-1-daf90a"

jaya@jaya-MacBook-Pro terraform %
```

2. Docker Run + Backend API Initialization

```
Terminal Local x Local (2) x Local (3) x + v
backend-api
[ec2-user@ip-10-20-3-128 backend]$ sudo docker run -d -p 3000:3000 --name backend-api -e DB_HOST="picoposts-v2-db.c7uk4uq089w4.us-west-1.rds.amazonaws.com" -e DB_USER="appuser" -e DB_NAME="picoposts" -e DB_PASSWORD="vhogHh8quKZGDeDpcV6H" backend-api
e84e983acd9c1024a0dc84b4d3d049ac95cde5b2c00a2ea51fb8af5a1dd108f
[ec2-user@ip-10-20-3-128 backend]$ sudo docker logs backend-api
Missing one of DB_HOST_WRITE / DB_USER / DB_NAME / DB_PASSWORD
[ec2-user@ip-10-20-3-128 backend]$ sudo docker stop backend-api
backend-api
[ec2-user@ip-10-20-3-128 backend]$ sudo docker rm backend-api
backend-api
[ec2-user@ip-10-20-3-128 backend]$ sudo docker run -d -p 3000:3000 \
> --name backend-api \
> -e DB_HOST_WRITE="picoposts-v2-db.c7uk4uq089w4.us-west-1.rds.amazonaws.com" \
> -e DB_HOST_READ="picoposts-v2-db.c7uk4uq089w4.us-west-1.rds.amazonaws.com" \
> -e DB_USER="appuser" \
> -e DB_NAME="picoposts" \
> -e DB_PASSWORD="vhogHh8quKZGDeDpcV6H" \
> backend-api
3cecab67773fc2187d94f7d37ebac8b33a2186c52723d27d38e51061391843da
[ec2-user@ip-10-20-3-128 backend]$ sudo docker logs backend-api
MySQL pool initialized
PicoPosts API listening on port 3000
[ec2-user@ip-10-20-3-128 backend]$
```


3. MySQL Table Creation + Curl Tests

```

Terminal Local x Local (2) x Local (3) x + v

Type 'help;' or 'h' for help. Type 'c' to clear the current input statement.

MySQL [(none)]> USE picoposts;
Database changed
MySQL [picoposts]> CREATE TABLE IF NOT EXISTS users (
  ->   id VARCHAR(255) PRIMARY KEY,
  ->   email VARCHAR(255),
  ->   created_at DATETIME
  -> );
Query OK, 0 rows affected (0.08 sec)

MySQL [picoposts]> CREATE TABLE IF NOT EXISTS posts (
  ->   id VARCHAR(255) PRIMARY KEY,
  ->   user_id VARCHAR(255),
  ->   content TEXT,
  ->   created_at DATETIME,
  ->   INDEX (user_id)
  -> );
Query OK, 0 rows affected (0.04 sec)

MySQL [picoposts]> \q
Bye
[ec2-user@ip-10-20-3-128 ~]$ curl -X POST \
> -H "Content-Type: application/json" \
> -d '{"email":"test@example.com"}' \
> http://picoposts-v2-alb-1318435385.us-west-1.elb.amazonaws.com/api/users
{"userId":"7034eeefb-f7d9-4020-9d0d-cb04c07bf8b2"}[ec2-user@ip-10-20-3-128 ~]$ curl -X POST \
> -H "Content-Type: application/json" \
> -d '{"userId":"7034eeefb-f7d9-4020-9d0d-cb04c07bf8b2","content":"hello from AWS"}' \
> http://picoposts-v2-alb-1318435385.us-west-1.elb.amazonaws.com/api/posts
{"postId":"ffabe954-f44d-4a4e-8862-fbba050b6486"}[ec2-user@ip-10-20-3-128 ~]$
[ec2-user@ip-10-20-3-128 ~]$ curl "http://picoposts-v2-alb-1318435385.us-west-1.elb.amazonaws.com/api/feed?userId=7034eeefb-f7d9-4020-9d0d-cb04c07bf8b2"
{"posts":[{"id":"ffabe954-f44d-4a4e-8862-fbba050b6486","content":"hello from AWS","created_at":"2025-12-07T01:00:29.000Z"}]}[ec2-user@ip-10-20-3-128 ~]$
[ec2-user@ip-10-20-3-128 ~]$
    
```

4. CloudFront – Signup Working

PicoPosts
Tiny social feed on AWS (EC2 + RDS + S3 + CloudFront)

Create user & post

Step 1: create a user. Step 2: write a tiny post.

EMAIL

jaya.vyas@sjsu.edu

We'll create a synthetic userid in the database for this email.

SIGN UP

USER ID

paste userid from above

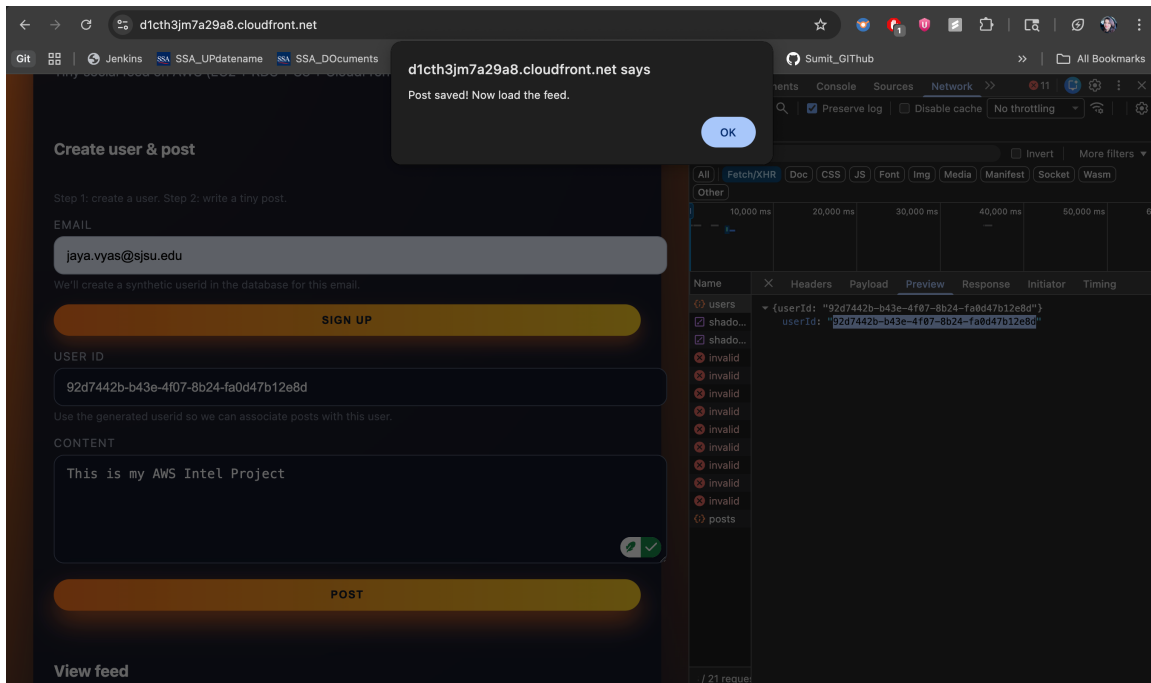
Use the generated userid so we can associate posts with this user.

CONTENT

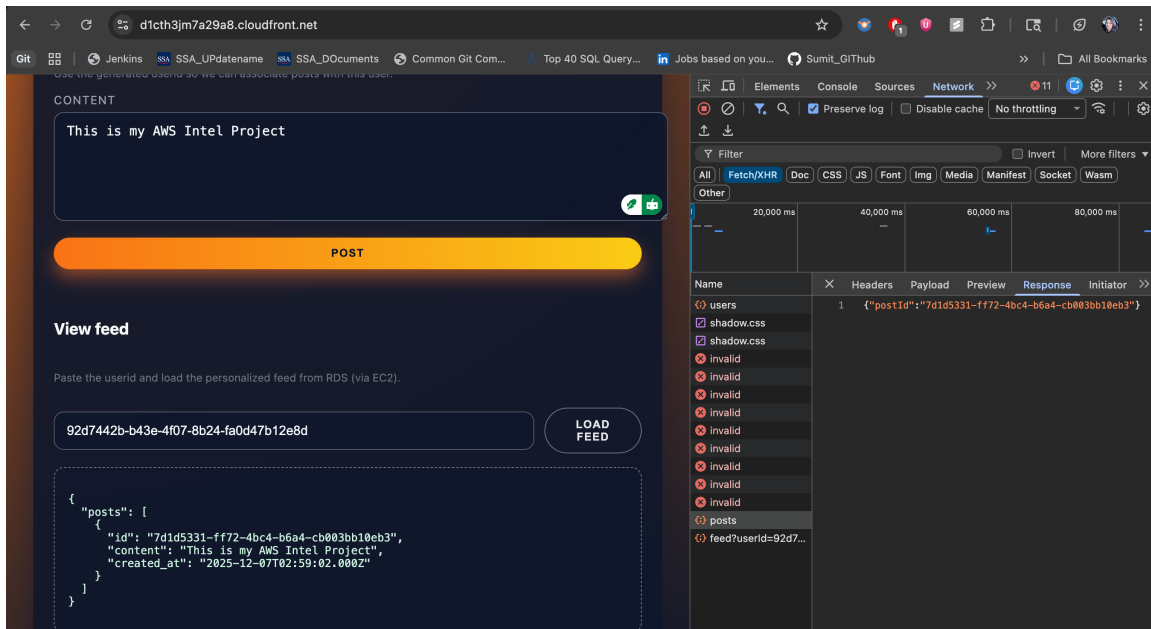
Notification: d1cth3jm7a29a8.cloudfront.net says
User created. User ID: 92d7442b-b43e-4f07-bb24-fa0d47b12e8d

Name	Status	Type	Initiator	Size	Time
users	200	fetch	requests.js:1	0.4 kB	88 ms

5. CloudFront – Post Creation Working



6. CloudFront UI Landing Page



7. Conclusion

This updated design applies enterprise-level cloud security practices, including least privilege, credential elimination, and robust IaC governance. The PicoPosts application now operates with reduced attack surface, improved reliability, and clearer operational processes.